

Comp 549: Code Analysis Assignment 1

February 7th, 2025

Bill Zimmerman 260981881

Question 1: Identifying the 8 aspects of a PDP

1. A set of processing units
 - a. This Model has 3 sets of units
 - i. 4 input units
 - ii. N hidden units
 - iii. 3 output units
2. A state of activation for each unit
 - a. The activated state in the hidden and output units are real numbers bounded between **0 and 1 (inclusive)**.
 - b. The state of activation for the units in both the hidden and output layers are represented by the following tensors:
 - i. Hidden Layer: `tensor0 = myfunction(np.dot(self.W0, helper(self.task.input)))`
 - ii. Output Layer: `tensor1 = myfunction(np.dot(self.W1, helper(tensor0)))`
3. An output function for each unit
 - a. The output function for the units in the hidden and output layers is the identity.
 - b. As stated in class, activation function and output function are used somewhat interchangeably in today's machine learning parlance. However, if the output level is exactly equal then the output function is the identity [1].
4. A pattern of connectivity between units
 - a. The units are fully connected. Each unit receives inputs from all the units in the layer behind it.
 - b. The pattern describing the connections between the units are defined by the weight matrices:
 - i. `self.W0`
 - ii. `self.W1`
5. A propagation rule for sending activity between units
 - a. The propagation rule is net input where each unit gets a weighted sum of the outputs of the neurons it is connected to.
 - b. These sums are performed by the dot products below
 - i. `np.dot(self.W0, helper(self.task.input))`
 - ii. `np.dot(self.W1, helper(tensor0))`

6. An activation rule for combining propagated activity with current activity.
 - a. The activation function is the linear saturating function. This function clips all values above 1 and below 0 to 1 and 0 respectively.
 - b. This function can be defined as:
 - i.
$$f(x) = \begin{cases} 0, & x < 0 \\ x, & 1 \geq x \geq 0 \\ 1, & 1 > x \end{cases}$$
 - c. This is represented by myfunction()
7. A learning rule
 - a. The learning rule, which alters connections based on experience, is backpropagation of error. This is done via a (pseudo) gradient descent algorithm which finds the direction of change for each weight that minimizes the difference between the correct output and predicted output. The reason for the pseudo is that the derivative of the activation is excluded from the chain rule. This is because below 0 and above 1 the derivative of the activation is 0. This can lead to vanishing gradients.
 - b. The gradients are calculated and represented by
 - i. `delta_W0 =`

$$\text{np.dot}(\text{np.dot}(\text{self.W1[:, :1]}.transpose(), \text{error}), \text{helper}(\text{self.task.inputs}).transpose())$$
 - ii. `delta_W1 = np.dot(error, helper(tensor0).transpose())`
 - c. The weights are updated in these lines
 - i. `self.W0 += self.eta*delta_W0`
 - ii. `self.W1 += self.eta*delta_W1`
8. An environment
 - a. The environment is a series discrete timesteps which each contain points on hypercubes. It is different to [1] which claims an environment is stochastic, this environment is deterministic, and the values are fed in fixed manner.
 - b. The environment is represented by the inputs tensors and the outputs tensor

Question 2: Describe in your own words what computation is being calculated and what algorithm is implemented to perform this calculation? How did you reach this conclusion?

◦ **Hint: Consider what space the units are operating in at each stage of processing.**

The function takes as input, a control bit and 3 coordinates representing a vertex on a cube and at the output layer, it returns a vertex on the cube, the same vertex if the control bit is 0, the opposite vertex if the control bit is 1. Opposite meaning point $(1,0,1) \rightarrow (0,1,0)$.

The hidden layer projects the point onto an n dimensional hypercube via matrix multiplication. The location in the n dimensional space the point is projected to depends on the control bit. Then point is projected back down to the same point or is projected down to the opposite vertex.

This needs to be done because “inverting” is a difficult concept to explain computationally. Imagine 3 planes that each bisect 2 parallel faces of a cube. Inverting a point requires one to map the point across each of these planes. This is not a linearly separable problem in 3 dimensions. Thus, we need to project to a higher dimension to make it linearly separable. In other words, projecting to a higher dimension allows us to find linear boundary across which we need to invert. Then we can project back down to 3 dimensions with our inverted point. This is known as Cover. A 2d-3d analogy is shown in figure 1.

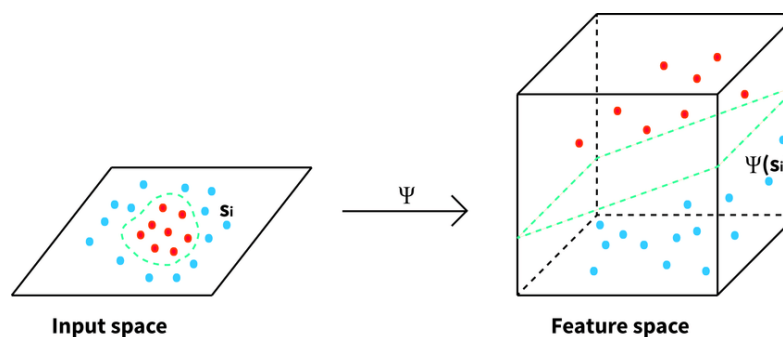


Figure 1 Visualization of how Projecting to Higher Dimensions Makes a Problem Linearly Separable [2]

Question 3: Can you predict what the algorithm will do with a novel input? ◦ Hint: Consider what a “soft” version of the computation would look like.

Assuming that the control bit always remains 1 or 0, the soft input would be a point on (or in) the cube. The output would be the point on (or in) the cube that has been inverted across the three axes that bisect the parallel faces. For example, if we got the input $(1, 0.2, 0.5, 0.7)$, the point inside $(0.2, 0.5, 0.7)$ would be inverted to the opposite $(0.8, 0.5, 0.3)$.

Question 4

How does the network's behaviour change as you change the value of N from very low to very high? Why would it change in this way?

The graphs below show that, in general, as N increases the error decreases faster and to a lower value. The reason for this is that N controls the number of hidden units, or the dimensionality we project up to. We know that as the number of dimensions increases the likelihood of finding local minima decreases. This means there are more paths towards the global minimum and meaning that the optimum can be reached faster. This is like how as we project to higher dimensions to create a more easily solvable problem.

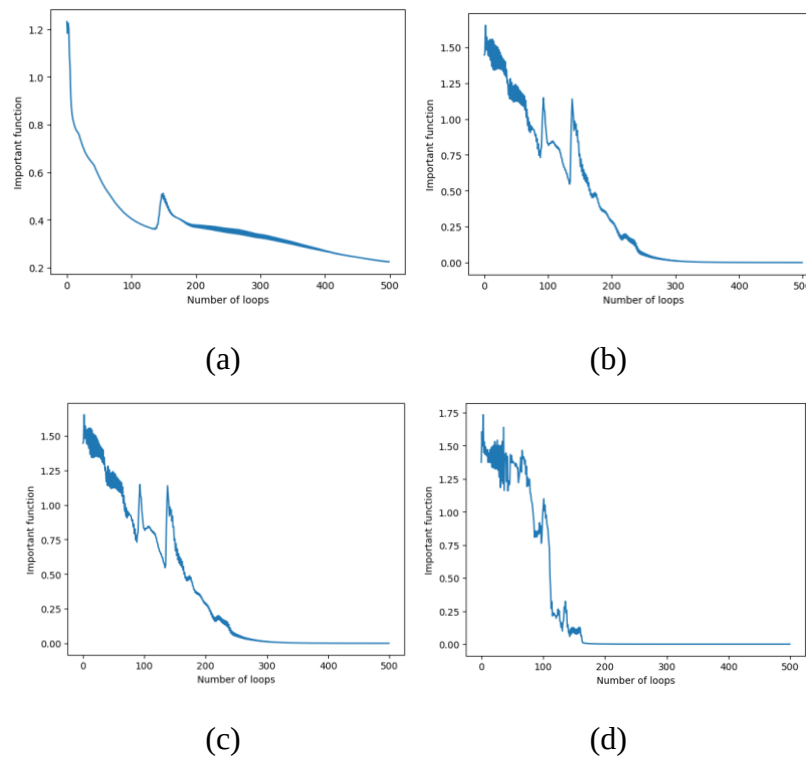


Figure 2 Loss function graph with $N = 10$ (a) 25 (b) 50 (c) and 100 (d)

What happens if you set eta to be very high? Why would this be the impact?

Eta is the learning rate used by our gradient descent function; it controls the scale by which we update our weights. As we increase eta, we increase the step size, and we see a slower and more erratic decrease in the value our loss function, if we even see one. We can also observe a higher plateau or a divergent function that oscillates or even increases.

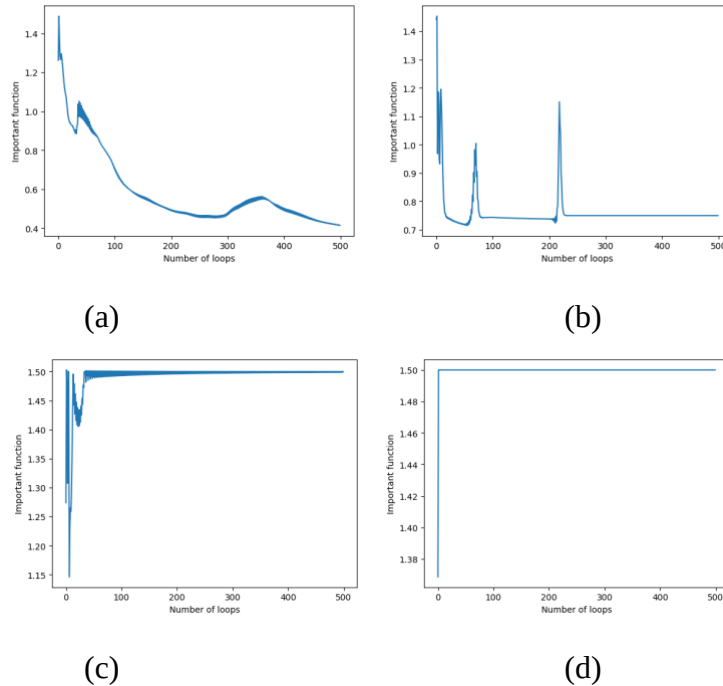


Figure 3 Loss Graph for Eta = 0.01 (a) 0.1 (b) .15 (c) and 1 (d)

This happens because as we increase our step size, we can update our weights too drastically and end up skipping over minima. This can be thought of like jumping between two sides of the minimum of a parabola. Even though we are moving in the direction of the minima each time, we overshoot it and end up quite far away.

What is the helper function doing? Why do we need it?

The helper function appends bias rows to the weight matrices. We need the bias so that the modeled function does not always pass through the origin. Figure 4 shows the need for this as if all our boundaries were forced to pass through the origin, certain classifications would become impossible.

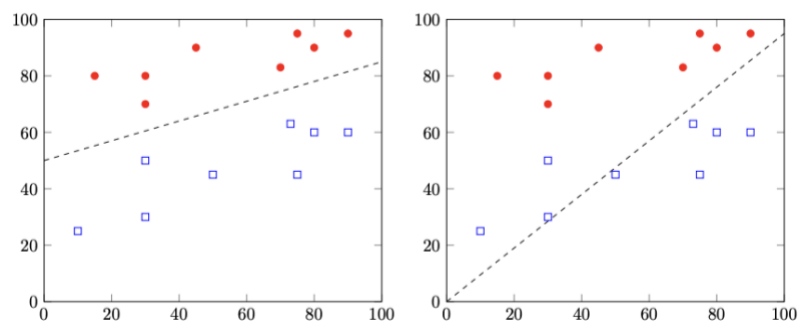


Figure 4 Learning Boundary for a Perceptron [3]

References

- [1] D. E. Rumelhart and J. L. McClelland, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*. Cambridge, MA: MIT Press, 1986.
- [2] Salvador, Matteo & Dede, Luca & Manzoni, Andrea. (2021). Non-intrusive reduced order modeling of parametrized PDEs by kernel POD and neural networks. 10.48550/arXiv.2103.17152.
- [3] W. Hamilton, *Comp 451 Fundamentals of Machine Learning*, “Chapter 3 – parametric learning and perceptrons,” cs.mcgill.ca, https://cs.mcgill.ca/~wlh/comp451/files/comp451_chap3.pdf (accessed Feb. 6, 2025).